

# 树链剖分

树链剖分是对树进行轻重链剖分，重链的条数不超过  $\log n$  条，用线段树维护链上信息，树链剖分可以支持的操作有：

- (1) 树链修改
- (2) 子树修改
- (3) 树链求和
- (4) 子树求和。

树链剖分可以解决大部分静态树上的各种修改查询问题，最坏时间复杂度为  $n \log n \log n$ 。这里说的静态指的是树的形态不发生改变，只改变结点信息。

但是对于修改树的形态的查询效率就比较低了，比如对于一棵树断掉一条边或者两棵树连成一棵树。所以就有了动态树这种东西 (link cut tree)。

# 动态树

至于为什么树链剖分不可以解决动态树上的问题呢？原因很简单，因为线段树不支持动态树的修改和查询。好好回忆一下，线段树的形态是不是一开始就是固定好了的？每次只会修改树的权值，也不会更改形态。

那么link cut tree既然能维护动态树的信息，那么他是会涉及到一种全新的数据结构吗？其实并不是，在我们已经学习过的数据结构中，有一种数据结构是可以维护一些基本的树的形态发生的变化的查询的——平衡树。平衡树为了维护平衡，树的形态需要一直发生改变。

所以lct实际上我们可以理解为树链剖分+平衡树(Splay)，当然这里的平衡树肯定是要支持区间修改的，所以一般用的Splay，FHQ写起来好像比较复杂，时间复杂度上最坏情况下会多一个log，不太推荐

# 动态树

先简单介绍一个动态树的实现思路，他也是一种剖分思想，对于树链剖分，他是先按照轻重链来剖分好，然后一条重链代表的编号连续，把树上一段区间拆分成多条链来查询。动态树也是剖分，但是他不是预先拆分好，而是需要时再拆分，比如我要查询 $x \rightarrow y$ ，那么我就把 $x \rightarrow y$ 这条路径拆分成一条独立的链(链起点为 $x$ ，终点为 $y$ )，那么这条链的总长度就是我们需要的答案。那么问题很明显，每次剖分之后，下次查询的一对点很有可能不再一条完整的链上，那么我们就需要来维护一下。首先介绍两个定义：

**实边：**用实边连接的点在一条链上，实链上的点可以通过父亲节点找儿子，也可以通过儿子节点找父亲。

**虚边：**用虚边连接的点不在一条链上，虚链上的可以通过儿子节点找父亲，不可以通过父亲节点找儿子。

# 动态树

对于每一条链，因为他的形态会随时发生改变，所以我们考虑用平衡树来维护，保证每次查询的  $(x, y)$  刚好在一条实链上。

所以动态树实际上是维护多棵平衡树，同时为了让平衡树的形态比较固定，层次分明，我们让一颗平衡树中的点深度按照中序遍历是递增的。即平衡树中维护的是一条深度递增的序列(就算不满足肯定可以通过旋转让他满足)

比如有一棵树，根节点为1（深度1），有两个儿子2,3（深度2），那么 Splay 有3种构成方式：

$\{1 - 2\}, \{3\}$

$\{1 - 3\}, \{2\}$

$\{1\}, \{2\}, \{3\}$ （每个集合表示一个 Splay）

而不能把1,2,3同放在一个 Splay 中（存在深度相同的点）

# 动态树

我们来思考一下，假设我们要求根节点到节点A的简单路径之和，那么很简单，我们只需要把根节点到节点A的路径上的点都放进一个splay就可以了，并且一定是可以的，因为很明显深度是递增的。

关键是怎么放呢？最开始肯定在不同的splay中。

其实也很简单，我们先把A变为当前splay的根，那么A的左子树是在这条路径上，右子树都不在，那么右子树就变为虚边。然后 $f[A]$ 就跳到了上一棵平衡树中，再按照前面的办法，把这个节点也变为当前splay的根，同样该节点的左子树都在路径上，右子树连接上一棵平衡树的根节点(A)。那么以前树中的另一个儿子就变成了虚边了。一直这样上去，我们就可以找到根节点所在平衡树，这样就把根节点到A节点放入一棵平衡树中来了。

# 动态树

那么如果求路径A-->B呢？，A,B都不是根节点，但是没关系啊，我把A旋转到根节点就可以了(总树的根节点)，然后再重复前面的操作。

```
void access(int x)//把x和根节点放入同一棵平衡树
{
    int y=0;
    while(x)
    {
        splay(x);
        tree[x].ch[1]=y;//最开始以x结尾，所以后面没有!
        //对于前面的平衡树，连接到下一棵平衡树的根节点
        update(x);
        y=x;//y表示下一棵平衡树根节点
        x=tree[x].f;//x跳到路径上一个点去
    }
}
```

# 动态树

当我们把根节点到节点 $x$ 的路径都放入一颗平衡树中之后，再把 $x$ 旋转变为平衡树的根（我们要求 $x \rightarrow y$ 的路径），但是这里有一个问题？？？

在这棵平衡树中， $x$ 的深度最深，那么此时树的形态是 $x$ 只有左儿子，没有右儿子。而我们前面需要的是保证根节点是深度最浅的点，这样才能让这棵平衡树按照深度递增找下去。该怎么办？翻转，翻转完之后根节点是不是变成了深度最浅的点了。然后再打通 $y$ 节点到 $x$ 节点的通道，然后再把 $y$ 节点旋转变为根节点，此时 $y$ 为根节点的平衡树就代表了路径 $x \rightarrow y$ 。

# 动态树

```
void makeroot(int x)
{
    access(x);
    splay(x);
    tree[x].lazy^=1; // 翻转, 保证x的深度在平衡树中最小
}
void split(int x, int y)
{
    makeroot(x); // 把x变为带根节点的平衡树的根节点, 旋转之后此时深度最小
    access(y); // 打通y到x的通道
    splay(y); // 把y旋转到当前平衡树的根
}
```



# 动态树

至于splay操作，emmm，常规操作，但是要注意的是因为我们是从下面节点向根节点旋转，所以先找到当前splay的根，把lazy标记传下去之后，再执行rotate操作。

```
int t=x,top=0;
st[++top]=t;
while(nroot(t))//如果不是当前平衡树根节点
{
    t=tree[t].f;
    st[++top]=t;//记录这些点
}
while(top)//这些点标记下传，只传修改的那一部分就可以了
{
    pushdown(st[top]);
    top--;
}
```

# 动态树

至于splay操作，emmm，常规操作，但是要注意的是因为我们是从下面节点向根节点旋转，所以先找到当前splay的根，把lazy标记传下去之后，再执行rotate操作。

```
int t=x,top=0;
st[++top]=t;
while(nroot(t))//如果不是当前平衡树根节点
{
    t=tree[t].f;
    st[++top]=t;//记录这些点
}
while(top)//这些点标记下传，只传修改的那一部分就可以了
{
    pushdown(st[top]);
    top--;
}
```

# 动态树

至于判断一个点是否是该平衡树根节点，很简单，前面已经说了认父亲不认儿子，并且两颗平衡树相连的边是虚边，那么说明该根节点的父亲节点的儿子不是他。

```
int nroot(int x)
{
    int y=tree[x].f;
    if(tree[y].ch[0]==x||tree[y].ch[1]==x)
        return 1;
    return 0;
}

while(nroot(x))
{
    int y=tree[x].f;
    int z=tree[y].f;
    if(nroot(y))
    {
        if((tree[z].ch[0]==y)^(tree[y].ch[1]==x))
            rotate(x);
        else
            rotate(y);
    }
    rotate(x);
}
```

# 动态树

```
void rotate(int x)
{
    int y=tree[x].f;
    int z=tree[y].f;
    int k=tree[y].ch[1]==x;
    if(nroot(y))//一定要判断, 否则会出现0节点的右儿子是x
    //那么x节点永远旋转不到根节点去, 死循环
    tree[z].ch[tree[z].ch[1]==y]=x;
    tree[x].f=z;
    tree[y].ch[k]=tree[x].ch[k^1];
    tree[tree[x].ch[k^1]].f=y;
    tree[x].ch[k^1]=y;
    tree[y].f=x;
    update(y);
    update(x);
}
```

# 动态树

如果需要连接两个点，先一定要把其中一个点 $x$ 旋根，再判断是否本来就连接，如果不连接，直接让另一个点 $y$ 的父亲指向 $x$ ，至于 $x$ 的儿子没有必要改变，因为改变了可能会影响平衡树的形态。

判断是否联通，我们可以先把 $x$ 旋转到根节点，再打通 $y$ 到 $x$ 的通道，把 $y$ 旋转到根节点，当然，如果不连通肯定打不通，此时 $x$ 是深度最深的点，然后再一直向左找，就会找到深度最深的点，那么就是该平衡树的根节点。

如果是删边，也是一样的，判断两个点是否直接相连，首先和上面一样判断联通性，再判断 $y$ 的父亲是否是 $x$ ，儿子就没有必要判断了，再判断 $y$ 是否有左儿子。

# 动态树

```
void link(int x,int y)
{
    makeroot(x);
    if(findroot(y)!=x)
        tree[x].f=y;
}
void cut(int x,int y)
{
    makeroot(x);
    if(findroot(y)==x&&tree[y].f==x&&tree[y].ch[0]==0)
    {
        tree[y].f=tree[x].ch[1]=0;
    }
}
```

# 动态树

```
int findroot(int x)
{
    access(x);
    splay(x);
    while(tree[x].ch[0])
    {
        pushdown(x);
        x=tree[x].ch[0];
    }
    splay(x); // 此时x还是原树的根
    return x;
}
```



# 动态树

洛谷 P2486 染色

洛谷 P3203 弹飞绵羊

洛谷 P1501 Tree 2

洛谷 P3721 单旋

洛谷 P2387 魔法森林

洛谷 P2173 网络

洛谷 P3950 部落冲突

